

18. Interpreter Design Pattern in Java

18.1 Introduction

The **Interpreter Design Pattern** is a **behavioral pattern** used to define a grammatical representation for a language and provide an interpreter to deal with this grammar.

18.2 Problem it Solves

You want to interpret or evaluate sentences in a language (e.g., a scripting language, a rule engine, a DSL).

18.3 What is Interpreter Pattern?

It defines a language's grammar and provides an interpreter to interpret sentences written in the language.

18.4 Real-World Analogy

A calculator that parses and evaluates mathematical expressions like $5 + 3 - 2$.

18.5 Structure

```
package com.mannemlokeskesh;  
  
public interface Expression {  
    boolean interpret(String context);  
}  
  
class TerminalExpression implements Expression {  
    private String data;  
    public TerminalExpression(String data) {  
        this.data = data;  
    }  
    public boolean interpret(String context) {  
        return context.contains(data);  
    }  
}  
  
class OrExpression implements Expression {  
    private Expression expr1, expr2;  
    public OrExpression(Expression expr1, Expression expr2) {  
        this.expr1 = expr1;  
        this.expr2 = expr2;  
    }  
    public boolean interpret(String context) {  
        return expr1.interpret(context) || expr2.interpret(context);  
    }  
}
```

18.6 Benefits

- Useful for designing simple scripting or rule languages
- Easy to extend the grammar

18.7 Implementation Steps

1. Define an Expression interface
2. Implement TerminalExpression and composite expressions like OrExpression, AndExpression, etc.
3. Build the abstract syntax tree (AST)
4. Evaluate input through the AST

18.8 Examples

Example1: Boolean Expression Interpreter

```
package com.mannemlokes.example1;

interface Expression {
    boolean interpret(String context);
}

class TerminalExpression implements Expression {
    private String keyword;

    public TerminalExpression(String keyword) {
        this.keyword = keyword;
    }

    public boolean interpret(String context) {
        return context.contains(keyword);
    }
}

class OrExpression implements Expression {
    private Expression expr1;
    private Expression expr2;

    public OrExpression(Expression expr1, Expression expr2) {
        this.expr1 = expr1;
        this.expr2 = expr2;
    }

    public boolean interpret(String context) {
        return expr1.interpret(context) || expr2.interpret(context);
    }
}

public class Example1InterpreterClient {
    public static void main(String[] args) {
        Expression male = new OrExpression(new TerminalExpression("John"),
                                           new TerminalExpression("Mike"));

        System.out.println("Is 'John' male? " + male.interpret("John"));
        System.out.println("Is 'Lisa' male? " + male.interpret("Lisa"));
    }
}
```

Output:

```
Is 'John' male? true
Is 'Lisa' male? false
```

Example2: Arithmetic Interpreter

```
package com.mannemlokes.example2;

interface Expression {
    int interpret();
}

class Number implements Expression {
    private int number;

    public Number(int number) {
        this.number = number;
    }

    public int interpret() {
        return number;
    }
}

class Add implements Expression {
    private Expression left, right;

    public Add(Expression left, Expression right) {
        this.left = left;
        this.right = right;
    }

    public int interpret() {
        return left.interpret() + right.interpret();
    }
}

class Subtract implements Expression {
    private Expression left, right;

    public Subtract(Expression left, Expression right) {
        this.left = left;
        this.right = right;
    }

    public int interpret() {
        return left.interpret() - right.interpret();
    }
}

public class Example2ArithmeticClient {
    public static void main(String[] args) {
        Expression result = new Subtract(new Add(new Number(10),
                                                    new Number(5)), new Number(3));
        System.out.println("Result: " + result.interpret());
    }
}
```

Output:

Result: 12

Example3: SQL WHERE Clause Interpreter

```

package com.mannemlokes.example3;

interface Expression {
    boolean interpret(String record);
}

class FieldContains implements Expression {
    private String keyword;

    public FieldContains(String keyword) {
        this.keyword = keyword;
    }

    public boolean interpret(String record) {
        return record.contains(keyword);
    }
}

class AndExpression implements Expression {
    private Expression expr1, expr2;

    public AndExpression(Expression expr1, Expression expr2) {
        this.expr1 = expr1;
        this.expr2 = expr2;
    }

    public boolean interpret(String record) {
        return expr1.interpret(record) && expr2.interpret(record);
    }
}

public class Example3SQLWhereClient {
    public static void main(String[] args) {
        Expression expr = new AndExpression(new
            FieldContains("department=HR"), new FieldContains("active=true"));

        String record1 = "department=HR,active=true";
        String record2 = "department=IT,active=true";

        System.out.println("Record1 Match: " + expr.interpret(record1));
        System.out.println("Record2 Match: " + expr.interpret(record2));
    }
}

```

Output:

```

Record1 Match: true
Record2 Match: false

```

18.9 Interpreter vs Other Patterns

Pattern	Purpose
Interpreter	Evaluate language grammar or expressions
Strategy	Switch between algorithms
Visitor	Separate algorithm from object structure

18.10 Pros and Cons



Pros

- Easy to extend grammar
- Useful for implementing DSLs or rule engines



Cons

- Complex grammars become hard to manage
- Can get slow for large inputs or many rules

18.11 When to Use / Not to Use

Use When:

- You need to interpret simple languages or commands
- Building a DSL or rule-based engine

Avoid When:

- Complex grammar (better use parser generators)
- Performance is critical

18.12 Summary

Feature	Description
Intent	Define a language grammar and interpret it
Use Case	Rule engines, mathematical evaluators, DSLs
Key Benefit	Easy to modify, extend, and interpret expressions

The **Interpreter Pattern** is perfect for creating small language interpreters, rule engines, and custom evaluators.